

Challenges & Opportunities with LLM-Assisted Visualization Retargeting

Luke S. Snyder*
University of Washington

Chenglong Wang†
Microsoft Research

Steven M. Drucker‡
Microsoft Research

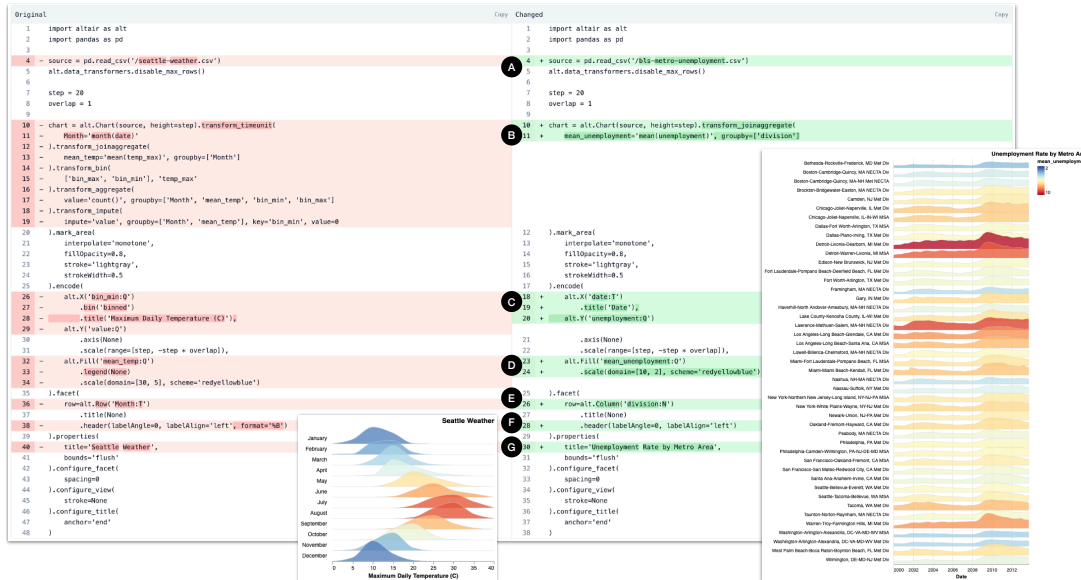


Figure 1: Manually retargeting an Altair ridgeline plot of Seattle weather data (left) to a new dataset containing United States unemployment rates across different metro areas (right). Summary of retargeting actions: (A) Providing the new dataset source; (B) Removing stale data transformations and replacing old data fields with new fields; (C-E) Updating new fields, domains, and titles for encoding channels; (F) Removing a stale formatter used for the month field; (G) Updating the chart title.

ABSTRACT

Despite the ubiquity of visualization examples published on the web, retargeting existing custom chart implementations to new datasets remains difficult, time-intensive, and tedious. The adaptation process assumes author familiarity with both the implementation of the example as well as how the new dataset might need to be transformed to fit into the example code. With recent advances in Large Language Models (LLMs), automatic adaptation of code can be achieved from high-level user prompts, reducing the barrier for visualization retargeting. To better understand how LLMs can assist retargeting and its potential limitations, we characterize and evaluate the performance of LLM assistance across multiple datasets and charts of varying complexity, categorizing failures according to type and severity. In our evaluation, we compare two approaches: (1) directly instructing the LLM model to fully generate and adapt code by treating code as text inputs and (2) a more constrained program synthesis pipeline where the LLM guides the code construction process by providing structural information (e.g., visual encodings) based on properties of the example code and data. We find that both approaches struggle when new data has not been appropriately transformed, and discuss important design recommendations

for future retargeting systems.

Index Terms: Visualization, Large Language Models, Retargeting

1 INTRODUCTION

Data visualizations are typically authored from scratch or adapted from extant designs. A variety of tools support the former, ranging from low-level APIs [5] and interfaces for manual design [29] to higher-level grammars [30, 31, 16] and toolkits [12, 36] for rapid construction. However, depending on the visualization task, analysis and communication goals, and designer expertise, authoring a visualization from scratch can be tedious, time-consuming, and difficult. Consequently, charting tools often showcase design galleries [5, 30, 31] which may serve as useful starting points from which the designer can adapt and refine based on their own data and goals [3].

We define *Visualization Retargeting* as the process of adapting an existing chart to a new dataset. Retargeting in the visualization community has typically focused on *restyling* image outputs of charts, such as Scalable Vector Graphics (SVG) [11, 14, 15] or bitmaps [28]. These methods rely on deconstruction heuristics that are limited by what can feasibly be inferred from image output alone [35, 9, 28, 22], and lack context regarding how the source data was manipulated or transformed. Recent advances in Large Language Models (LLMs) open the possibility of improving this restyling effort on code (rather than image output) by being able to reason about the semantics and context of the new data and how it might be appropriately fit to an existing visual design. Critically, we explore the broader capabilities of LLM-assisted retargeting with code implementations of charts, encompassing not only possible restyling, but also transformations and mappings of new data (e.g.,

*e-mail: snyderl@cs.washington.edu

†e-mail: chenglong.wang@microsoft.com

‡e-mail: sdrucker@microsoft.com

filtering, aggregation), reuse and generation of visual scales and channels (e.g., scale domains, legends), and modification of axis and chart guides (e.g., titles, labels).

The primary contributions of this paper are threefold: (1) we outline and characterize the challenges that arise when retargeting an implemented visualization specification to new data; (2) we evaluate the performance of LLM-assisted retargeting across a variety of datasets and charts; and (3) we summarize opportunities and design recommendations for future retargeting systems to mitigate existing pitfalls with LLM assistance. Our evaluation tests both an LLM baseline and LLM program synthesis pipeline that guides the code generation and refinement process by extracting salient metadata (e.g., visual encodings and transforms), providing additional context to aid reasoning. We find that the pipeline performs worse than the baseline, possibly due to propagating errors, and that both approaches are unable to appropriately retarget most charts unless the data has been adequately transformed and filtered in advance.

2 RELATED WORK

2.1 Chart Deconstruction & Restyling

Prior work in chart deconstruction primarily aims to reverse-engineer data from an SVG or bitmap visualization, which may be used to support interactive features [35], restyling and redesign [14, 15], or layout reuse [9, 11]. Deconstruction algorithms typically rely on heuristics that are limited in scope, such as rectangle charts [9] or infographics [11], to ease inference. Computer vision algorithms may also be used for bitmap images [28], but are error-prone in certain cases (e.g., overlapping marks [22]) without the additional structure and context that SVG format provides.

Our work draws on chart deconstruction methods by attempting to guide LLMs in detecting data mappings, scales, and encodings included in a chart specification. This information provides necessary scaffolding to then map to a new dataset. Given the wide range of possible errors with reverse-engineering a chart image, we instead focus on the chart's *specification* format (e.g., Vega-Lite [30] grammar representation or Matplotlib Python code). Specifications also provide important context that would be impossible in many cases to infer from an image alone, such as complex data transformations (e.g., aggregation, filtering, etc.).

2.2 Visualization Examples & Automated Support

Charts may be automatically constructed with smart defaults that depend on data types and semantics (e.g., Show Me [21], Vega-Lite [30]), although additional effort is required for customization. Visualization examples and online templates are useful starting points for designers to adapt more complex designs to their own data, but take considerable time and care and require adequate familiarity with the tool [3]. To aid this process, Shen et al. [34] proposed an automated pipeline to decouple Vega-Lite examples from their data source, which could then be modified with data fields from a new dataset. We apply this line of work to LLMs, which enables broader coverage of visualizations across different tools and can support more complex retargeting operations (e.g., data transformation, code generation, etc.).

3 CHALLENGES WITH VISUALIZATION RETARGETING

Retargeting visualization code to a new dataset involves multiple decisions about how to both modify the chart specification itself and refine the visual design based on data semantics. Using Fig. 1 as an example, we identify and characterize different tiers of retargeting challenges across Kosslyn's [20] categories for chart design and comprehension: syntax, semantics, and pragmatics.

Syntax. Syntax challenges when modifying code arise due to dependencies (e.g., variables, data fields) within the specification. For example, when retargeting the Altair ridgeline plot (Fig. 1), the designer must replace the dataset location (Fig. 1(A)) and data field

names (Fig. 1(B-E)). While these modifications are simple substitutions, more complex syntax changes are required when removing or updating data transformations or encodings. For instance, when the designer chooses to encode `date` on the x-axis instead of `bin_min` (Fig. 1(C)), transformations that derive `bin_min` (Fig. 1(B)) are rendered stale and should be removed. However, removing these transformations affects the derived field value that is used to encode the y-axis, which must then be replaced with another available field from the new dataset (Fig. 1(C)). Additionally, the decision to facet by `division` in the new dataset (Fig. 1(E)) also makes the `transform.timeunit` derivation obsolete (Fig. 1(A)).

Often, updating an encoding channel with a new data field also necessitates checking the formatting syntax, which may depend on the data type. For example, replacing `Month` with `division` (Fig. 1(E)) requires the author to update the type information as well: `Month:T` (for temporal) to `division:N` (for nominal). In addition, the formatting option for `month` (`format='%B'`) must be removed (Fig. 1(F)) for the visualization to render correctly with the categorical type of `division`. Given that these dependencies vary between tools, LLMs can ostensibly help generate code that not only accounts for the typing and semantics of the new data, but is also syntactically correct for the charting tool at hand (i.e., gulf of execution [18]).

Semantics. Successfully retargeting a visualization must also consider data semantics. Such decisions involve how new data fields should be mapped to fields from the old dataset, along with appropriate choices for encoding channels, axis and legend scales, and data transformations. For instance, `division` is mapped to the facet encoding (Fig. 1(E)) given that no other encodings represent discrete data; `date` is mapped to the x-axis (Fig. 1(C)) to illustrate unemployment changes over time; and `unemployment` is mapped to the color legend (Fig. 1(D)), which would be inappropriate for the other fields.

After mapping new data fields to encoding channels, the designer must also refine any scales or transformations based on semantic overlap. For example, the extent of the `mean_unemployment` differs from that of the `mean_temp` in the Seattle weather dataset, which must be reflected in the scale domain (Fig. 1(D)). While the color scale does not need to be modified in this case, there are situations in which it might (e.g., the new data field exhibits such a large variance that a logarithmic scale would be more appropriate). In addition, the designer can reuse the aggregate transform for the `division` field (Fig. 1(B)), given that it represents discrete data.

Finally, the designer must also update axis labels / titles (Fig. 1(C)) and chart titles (Fig. 1(G)) to reflect the semantics and context of the new dataset.

Pragmatics. A retargeted visualization should also consider if the appropriate amount of information is conveyed for the target audience, as well as visual aesthetics. For example, while a legend is not necessary for the original Altair ridgeline plot (Fig. 1(left)) given that it overlaps in context with the maximum daily temperature (x-axis), this assumption does not hold for the retargeted version (Fig. 1(right)), which should include a legend (Fig. 1(D)) to help the reader understand what color hue represents.

Other pragmatic decisions may include whether axis units should be included, if enough spacing is provided to prevent mark occlusion and ensure legibility, where text and legends should be positioned, how axis labels should be formatted, or how the chart should be modified for different modalities (e.g., responsive visualization for smaller screens [17]). For charts with communicative aids such as annotations, decisions must be made as to whether they should be removed or not. In this work, we primarily focus on supporting the LLM's analysis and refinement of syntax and semantics, given that pragmatic decisions are more difficult to automate and significantly depend on variable external factors (e.g., audience, communication / analysis goals, context of viewing, visual aesthetic choices, etc.).

Error Class	Type	Severity	Baseline (%)	Pipeline (%)
Syntactic	Data transformation not valid	High	21.88	18.75
	Old data field(s) not removed	High	0.0	28.13
	Library import removed	High	0.0	3.13
Semantic	Incorrect / inappropriate scales	Medium	16.0	12.5
	Incorrect axis label(s)	Medium	4.0	6.25
	Non-standard data encoding	Medium	20.0	25.0
	Annotation not removed	Medium	16.0	18.75
	Missing necessary transformation	Medium	32.0	31.25
	Incorrect or ambiguous axis title	Medium	12.0	6.25
Pragmatic	Overplotting	High	60.0	75.0
	Unnecessary use of faceting	Low	4.0	0.0
	Axis title missing units	Medium	20.0	25.0
	Heavy whitespace	Low	4.0	0.0
	Repeated axis labels	Low	4.0	6.25
	Axis labels not rotated	Medium	0.0	12.5
	Data encoding not necessary	Medium	4.0	0.0
	Missing axis titles	Medium	4.0	0.0
	Missing legend title	Medium	8.0	6.25

Table 1: Retargeting errors by type and prevalence (%) for both LLM baseline and pipeline approaches, normalized by rendered charts.

4 EVALUATING LLM-ASSISTED RETARGETING

4.1 Methods

We evaluate two strategies of LLM assistance. The first is a baseline approach that queries an LLM with minimal context consisting of only the chart specification and summary of the new dataset. The second is an LLM program synthesis pipeline that guides programmatic code generation with salient chart information (e.g., visual encodings, types of transformations, etc.) to enhance reasoning and provide useful context, mitigating existing design challenges (§3). This pipeline consists of three sequential stages: syntax parsing, data mapping, and specification refinement.

Syntax Parsing. We parse the chart specification into its Abstract Syntax Tree (AST), enabling automated detection and analysis of dependencies that occur when deriving or transforming new variable names [40, 13]. We then query the LLM to identify semantic roles of different variables and code sections, such as encoding channels (e.g., axes, legends), data fields, and transformations.

Data Mapping. We provide the LLM with both a summary of the new dataset (e.g., statistics, typing) and semantic roles from the syntax parsing stage to help it determine how the new dataset should be mapped to existing encodings (e.g., based on typing and semantic similarity) and appropriately transformed.

Specification Refinement. Finally, we query the LLM with the new data mappings and code dependencies from the syntax parsing stage to assist with code generation and adaptation: removing stale transformations and encoding channels, as well as updating titles, axis labels (e.g., units, formatting), encoding scales, and basic aesthetics (e.g., filtering or aggregating data to prevent overplotting).

We evaluated both the LLM baseline and pipeline across 4 datasets [2, 7, 6, 19] and 8 Python charts [26, 24, 25, 27, 32, 23, 1, 33] of varying complexity (different charting tools, chart types and encoding decisions, communication aids such as annotations, and data transformations), resulting in 64 retargeted visualizations (2 conditions $\times$ 4 datasets $\times$ 8 charts). We used gpt-4o as the LLM.

4.2 Results

To evaluate the performance of retargeting output, we open-coded errors across each of the 64 charts, noting both syntactic issues with the specification as well as semantic and aesthetic errors in the visual output. Specifically, the first author began by manually inspecting each retargeted specification and its rendered chart to extract fine-grained errors. Throughout this process, syntactic, semantic, and pragmatic axes (§3) were used as thematic aids to help consider different dimensions of errors for completeness. For example, the first author analyzed runtime, compilation, and linting errors to extract syntactic failures. They then analyzed visual components

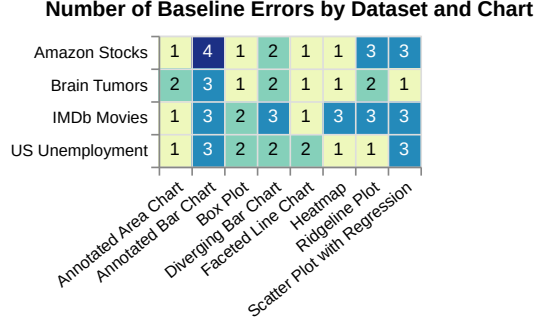


Figure 2: Baseline retargeting errors across 32 different chart instances. Higher errors are observed for more complex charts (annotations, regression) and datasets with high (IMDb) or low (US Unemployment) complexity.

of the rendered chart (marks, axes, scales & encodings, legends & titles, etc.) to identify ambiguous or inappropriate semantics and reflect on the pragmatics of the design (legibility, formatting, positioning, unnecessary or missing chart information, etc.). The first author coded the extracted error data to surface common types of errors (Table 1). Initial codes were refined and merged via pilot-testing with a subset of the data to ensure reliability. In addition, we noted the severity of an error type if it prevented the retargeted chart from rendering or it was not possible to reasonably interpret what was being displayed (e.g., significant overplotting due to lack of necessary transformations such as filtering or aggregation).

The results of our analysis are provided in Table 1. Out of the 32 chart instances per condition, 25 rendered in the baseline approach, but only 16 rendered with the LLM pipeline. Charts that failed to render were the result of syntactic errors, making it difficult or impossible to identify many of the downstream semantic or pragmatic decisions (e.g., Fig. 3(F)). As a result, we normalized all semantic and pragmatic errors by the total number of rendered charts. We found that the number of semantic and pragmatic errors for the pipeline were comparable to that of the baseline. Both approaches were unable to adequately transform data or remove certain encodings altogether to prevent overplotting (e.g., Fig. 3(B,C,E)). Further, the pipeline generated noticeably more syntactic errors, likely due to inference failures in the syntax parsing stage. If the LLM fails to appropriately infer semantic roles, this information may be incorrect or incomplete, complicating subsequent queries in the pipeline.

To determine relationships between retargeting performance and the characteristics of the datasets and charts, we plotted a heatmap with the counts of errors (Fig. 2). We excluded counts from the LLM pipeline given its inability to render a noticeably larger number of charts compared to the baseline. We found that the number of errors increase for more complex charts, such as annotations or multiple layers (e.g., regression lines as in Fig. 3(D)). In addition, both simple and complex datasets struggle with retargeting. For instance, the IMDb movies dataset [19] contains multiple columns that may require custom transformations prior to successful encoding. As shown in Fig. 3(E), although the movie genre is an appropriate fit to replace species in the original plot (Fig. 3(D)), this mapping requires necessary transforms to unpack the arrays. Compared with other datasets, the US unemployment data [7] contained the fewest number of columns, each of distinct type. Datasets without sufficient complexity (i.e., not enough data fields or types) can necessitate custom transformations to derive new fields or deleting complex code dependencies to reduce the number of encodings.

4.3 Design Recommendations

Based on the performance of LLM-assisted retargeting, as well as the type and severity of different errors, we discuss our recommendations for future retargeting systems.

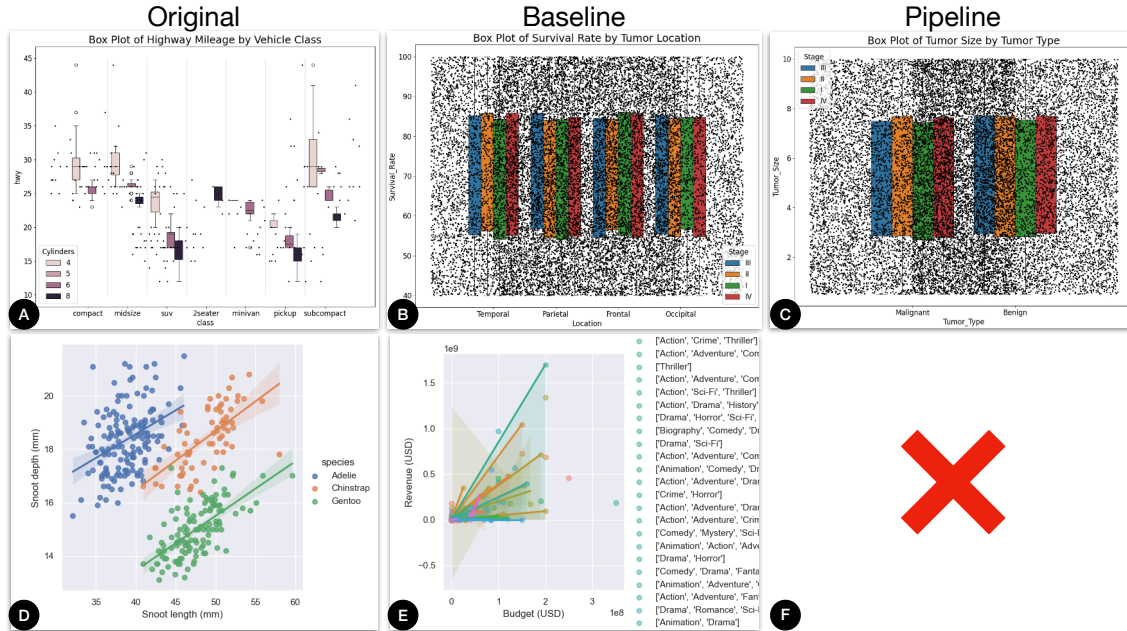


Figure 3: Example retargeted chart specifications for both LLM baseline and pipeline. (A-C) Retargeted Matplotlib box plot to brain tumor dataset. (D-F) Retargeted Seaborn scatter plot with regression to IMDb Movies dataset; the LLM pipeline fails to render the chart (F).

DR1: Enable Mixed-Initiative Assistance at Inflection Points.

Fig. 2 reveals that at least one error occurred with each instance of retargeting. An obvious result is that mixed-initiative interactions and interfaces will be necessary for some time to steer the LLM through the retargeting process (as with chart deconstruction tools [22]). However, the inflection points for user interaction and refinement will be different according to the author's retargeting goals, dataset and specification complexity, and types of errors that may be encountered. We expect the syntactic, semantic, and pragmatic axes for retargeting challenges (§3) to serve as useful stages for users to interact with intermediate LLM output, mitigating cascading errors and restricting the type of code that is generated.

DR2: Surface Data Dependencies. Despite the LLM pipeline's relatively weak performance, surfacing dependencies between data fields, transformations, and encodings from the syntax parsing phase could allow designers to inspect and refine inference errors. These corrections may help reduce subsequent semantic and pragmatic errors such as missing transformations or failing to update scale domains that use hard-coded values (Fig. 1(D)). Further, operating on AST representations may help isolate portions of the specification that should not be changed without explicit approval (e.g., specific transforms or encodings that are still desired), counteracting hallucinations or incorrect assumptions about the data.

DR3: Integrate Transformation Support. Prior work has explored LLM-assisted transformations with data tables to rapidly construct charts [38, 37], which generate precise scripts to manipulate and derive data columns. However, transformation decisions in visualization retargeting are often less flexible since they are constrained by possible mappings between data fields from the old and new datasets. As a result, future retargeting systems may benefit more from user-guided heuristics or flags that ease transformation inference. For example, despite being prompted to consider filtering and aggregation transformations, overplotting persisted with both the LLM baseline (Fig. 3(B)) and pipeline (Fig. 3(C)). Rather than generating a custom, one-off transformation script, the designer might be able to set explicit, reconfigurable flags on the retargeted output that coerces alignment with specific design constraints (e.g., filter / aggregate the data, remove an encoding, facet, etc.).

5 LIMITATIONS & FUTURE WORK

Image-based retargeting approaches (e.g., using vision models to analyze and manipulate SVG or bitmap images) might detect and correct visually salient errors (e.g., overlapping marks) with greater accuracy. Curated corpora [8] can provide high-quality images from a large set of diverse charting tools for evaluation. However, retargeting code provides other benefits: access to data transforms, portability of interactions and accessibility aids, and transparent retargeting edits via code diffs. Rather, we expect that using code and image output together can leverage the benefits of both. This approach can augment low-resource coding languages or new charting libraries that LLMs have not been adequately trained on. Notational design [4] (e.g., viscosity, hidden dependencies, abstraction) will likely affect performance as well (e.g., more complex viscous languages that require many changes to achieve one retargeting goal).

Newer reasoning models have been released since the start of this work (e.g., Claude 3.7 Sonnet [10]). Although these models may reason more effectively about data dependencies and transformations, we expect that our analysis of retargeting errors can help characterize performance differentials between models and support future benchmarking. Syntactic and semantic errors may occur less frequently as models mature in their coding and reasoning abilities (e.g., chain-of-thought [39]). However, pragmatic mistakes are likely to persist given their reliance on user input and shifting analysis contexts (e.g., tasks, goals, etc.). For instance, certain encodings or annotations, unit abbreviations / axis labels, and titles may or may not be necessary depending on the user's background and downstream viewing context (e.g., analysis vs. communication).

Our evaluation and analysis underscore the need for new interfaces and interaction paradigms to aid visualization retargeting. These systems must balance the need between ensuring correct retargeting behavior and involving the designer where and when it is appropriate. In particular, designers may lack familiarity with specific charting tools that could make it difficult for them to recognize syntactic or semantic errors in the specification, or know how to correct errors they have identified. One possible solution might be to sequence retargeting operations deliberately, helping users isolate specific changes and relate them with the image output.

ACKNOWLEDGMENTS

This work was mostly performed during an internship at Microsoft. We thank the VIDA team at Microsoft Research for their thoughtful feedback and support.

REFERENCES

- [1] https://altair-viz.github.io/gallery/ridgeline_plot.html. 3
- [2] <https://www.kaggle.com/datasets/varpit94/amazon-stock-data>. 3
- [3] H. K. Bako, X. Liu, L. Battle, and Z. Liu. Understanding how designers find and use data visualization examples. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):1048–1058, 2022. 1, 2
- [4] A. F. Blackwell, C. Britton, A. Cox, T. R. G. Green, C. Gurr, G. Kadoda, M. S. Kutar, M. Loomes, C. L. Nehaniv, M. Petre, C. Roast, C. Roe, A. Wong, and R. M. Young. Cognitive dimensions of notations: Design tools for cognitive technology. In M. Beynon, C. L. Nehaniv, and K. Dautenhahn, eds., *Cognitive Technology: Instruments of Mind*, pp. 325–341. Springer Berlin Heidelberg, 2001. 4
- [5] M. Bostock, V. Ogievetsky, and J. Heer. D³: Data-driven documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, 2011. 1
- [6] <https://www.kaggle.com/datasets/miadul/brain-tumor-dataset>. 3
- [7] <https://www.bls.gov/>. 3
- [8] C. Chen, H. K. Bako, P. Yu, J. Hooker, J. Joyal, S. C. Wang, S. Kim, J. Wu, A. Ding, L. Sandeep, A. Chen, C. Sinha, and Z. Liu. Visanatomy: An svg chart corpus with fine-grained semantic labels. *arXiv preprint arXiv:2410.12268*, 2015. 4
- [9] C. Chen, B. Lee, Y. Wang, Y. Chang, and Z. Liu. Mystique: Deconstructing svg charts for layout reuse. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):447–457, 2023. 1, 2
- [10] <https://www.anthropic.com/news/claude-3-7-sonnet>. 4
- [11] W. Cui, J. Wang, H. Huang, Y. Wang, C.-Y. Lin, H. Zhang, and D. Zhang. A mixed-initiative approach to reusing infographic charts. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):173–183, 2021. 1, 2
- [12] S. Drucker and R. Fernandez. A unifying framework for animated and interactive unit visualizations. *Microsoft Research*, 1, 2015. 1
- [13] K. Gu, R. Shang, R. Jiang, K. Kuang, R.-J. Lin, D. Lyu, Y. Mao, Y. Pan, T. Wu, J. Yu, et al. Blade: Benchmarking language model agents for data-driven science. In *Empirical Methods in Natural Language Processing*, 2024. 3
- [14] J. Harper and M. Agrawala. Deconstructing and restyling d3 visualizations. In *Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology*, pp. 253–262, 2014. 1, 2
- [15] J. Harper and M. Agrawala. Converting basic d3 charts into reusable style templates. *IEEE Transactions on Visualization and Computer Graphics*, 24(3):1274–1286, 2017. 1, 2
- [16] J. Heer and D. Moritz. Mosaic: An architecture for scalable & interoperable data views. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):436–446, 2023. 1
- [17] J. Hoffswell, W. Li, and Z. Liu. Techniques for flexible responsive visualization design. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, pp. 1–13, 2020. 2
- [18] E. L. Hutchins, J. D. Hollan, and D. A. Norman. Direct manipulation interfaces. In *User Centered System Design*, pp. 87–124. CRC Press, 1986. 2
- [19] <https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows>. 3
- [20] S. M. Kosslyn. Understanding charts and graphs. *Applied Cognitive Psychology*, 3(3):185–225, 1989. 2
- [21] J. Mackinlay, P. Hanrahan, and C. Stolte. Show me: Automatic presentation for visual analysis. *IEEE Transactions on Visualization and Computer Graphics*, 13(6):1137–1144, 2007. 2
- [22] D. Masson, S. Malacria, D. Vogel, E. Lank, and G. Casiez. Chart-detective: Easy and accurate interactive data extraction from complex vector charts. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, pp. 1–17, 2023. 1, 2, 4
- [23] <https://python-graph-gallery.com/best-python-chart-examples/>. 3
- [24] <https://matplotlib.org/stable/tutorials/lifecycle.html>. 3
- [25] <https://www.machinelearningplus.com/plots/top-50-matplotlib-visualizations-the-master-plots-python/>. 3
- [26] <https://www.machinelearningplus.com/plots/top-50-matplotlib-visualizations-the-master-plots-python/>. 3
- [27] <https://plotly.com/python/facet-plots/>. 3
- [28] J. Poco and J. Heer. Reverse-engineering visualizations: Recovering visual encodings from chart images. *Computer Graphics Forum*, 36(3):353–363, 2017. 1, 2
- [29] D. Ren, B. Lee, and M. Brehmer. Charticulator: Interactive construction of bespoke chart layouts. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):789–799, 2018. 1
- [30] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, 2016. 1, 2
- [31] A. Satyanarayan, R. Russell, J. Hoffswell, and J. Heer. Reactive vega: A streaming dataflow architecture for declarative interactive visualization. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):659–668, 2015. 1
- [32] <https://seaborn.pydata.org/generated/seaborn.heatmap.html>. 3
- [33] https://seaborn.pydata.org/examples/multiple_regression.html. 3
- [34] L. Shen, E. Shen, Z. Tai, Y. Wang, Y. Luo, and J. Wang. Galvis: visualization construction through example-powered declarative programming. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pp. 4975–4979, 2022. 2
- [35] L. S. Snyder and J. Heer. Divi: Dynamically interactive visualization. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):403–413, 2023. 1, 2
- [36] C. Stolte, D. Tang, and P. Hanrahan. Polaris: a system for query, analysis, and visualization of multidimensional databases. *Communications of the ACM*, 51(11):75–84, 2008. 1
- [37] C. Wang, B. Lee, S. M. Drucker, D. Marshall, and J. Gao. Data formulator 2: Iterative creation of data visualizations, with ai transforming data along the way. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pp. 1–17, 2025. 4
- [38] C. Wang, J. Thompson, and B. Lee. Data formulator: Ai-powered concept-driven visualization authoring. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):1128–1138, 2023. 4
- [39] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022. 4
- [40] L. Xie, C. Zheng, H. Xia, H. Qu, and C. Zhu-Tian. Waitgpt: Monitoring and steering conversational llm agent in data analysis with on-the-fly code visualization. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–14, 2024. 3